



ТЕСТИРОВАНИЕ DOM-MODEЛИ

Защита клиентской логики современного веба

Эволюция угроз: от сервера к клиенту

DOM (Document Object Model) — программное представление веб-страницы в браузере.

Революция: Современные SPA (React, Angular, Vue) переносят критичную бизнес-логику и рендеринг на сторону клиента.

Ключевая проблема: Традиционные сканеры анализируют только статический HTML-ответ сервера, но не видят динамические изменения DOM, созданные JavaScript.

Слепое пятно: Если уязвимость (XSS, логическая ошибка) существует только в клиентском коде и проявляется при манипуляциях с DOM, классические методы тестирования ее не найдут.

Уязвимости, рожденные в браузере

Определение: Уязвимости, источником которых является небезопасная обработка данных, управляемых клиентом (контролируемых атакующим), внутри DOM.

Отличие от отраженного/хранимого XSS:

- Обычный XSS: Вредоносный код внедряется в ответ сервера.
- DOM-based XSS (и другие): Вредоносный код формируется и выполняется исключительно на клиенте через взаимодействие с DOM. Сервер может быть «ЧИСТЫМ».

Уязвимости, рожденные в браузере

Категории DOM-уязвимостей:

- DOM-based XSS (самая распространенная).
- DOM Clobbering (загрязнение DOM).
- Уязвимости клиентской логики (например, обход проверок цен в корзине).
- Утечки данных через глобальные объекты.

Как возникает и работает?

Условия: В приложении есть JavaScript-код, который берет данные, контролируемые пользователем (например, из `document.location.hash`), и небезопасно передает их в функцию-источник.

Источники: `document.URL`, `document.location`, `document.referrer`, `window.name`.

Стоки (Sinks): Опасные функции/свойства, которые выполняют код или изменяют

DOM: `innerHTML`, `outerHTML`, `document.write()`, `eval()`, `setTimeout()`, `location.assign()`.

Пошаговый подход

Разведка:

- Ручной обход приложения с открытыми Инструментами разработчика.
- Просмотр вкладок Sources и Network для анализа клиентских JS-файлов.
- Ключевой инструмент: Вкладка Inspector для просмотра текущего состояния DOM.

Пошаговый подход

Анализ источников:

Поиск в коде упоминаний `location`, `URL`, `hash`, `search`, `referrer`, `postMessage`, `localStorage`. Отслеживание, куда передаются эти данные.

Пошаговый подход

Отслеживание потока данных:

Мысленно или с помощью инструментов отследить путь данных от источника до потенциального стока.

Поиск санитизации/валидации: проверяется ли ввод? Используется ли кодирование?

Пошаговый подход

Проверка стоков:

Подстановка тестовых векторов (например, ">") в найденные источники и наблюдение за реакцией DOM/JS.

Арсенал тестировщика

Браузерные (основа):

Debugger: Пошаговое выполнение кода, точки останова.

Console: Интерактивное выполнение JS, проверка объектов.

Search (Ctrl+Shift+F): Поиск по всем JS-файлам (innerHTML, eval, location.hash).

Арсенал тестировщика

Продвинутые инструменты анализа потока данных:

[Burp Suite's DOM Invader](#): Автоматически находит источники и стоки, подсказывает векторы для DOM XSS и Clobbering.

[OWASP ZAP](#) (клиентские скрипты): Позволяет писать скрипты для модификации запросов/ответов на лету.

[tarn.js](#), [Jalangi2](#): Академические инструменты для статического и динамического анализа потока данных в JS.

Арсенал тестировщика

Браузеры для тестирования: Chrome/Edge (лучшая DevTools), Firefox.

За пределами простого XSS

DOM Clobbering: Атакующий внедряет в DOM безопасные HTML-элементы (например, `<a id="config">`), которые «затирают» важные JS-переменные, влияя на логику приложения. Часто используется для обхода CSP или усиления XSS.

Уязвимости в сторонних библиотеках: Уязвимый компонент npm (например, jQuery старой версии) используется в вашем SPA. Тестирование должно включать анализ зависимостей.

Утечки через глобальный объект: небезопасное расширение `Object.prototype` может привести к утечке данных в консоли или к ошибкам.

PostMessage-атаки: Некорректная обработка сообщений между окнами/iframe может привести к XSS или утечке данных.

Как разрабатывать безопасные клиентские приложения?

1. Избегайте опасных стоков:

Вместо `innerHTML/outerHTML` -> используйте `textContent` или безопасные API (`document.createElement`, `appendChild`).

Никогда не используйте `eval()`, `setTimeout()` со строкой, `new Function()` с данными пользователя.

Как разрабатывать безопасные клиентские приложения?

2. Контекстно-зависимое кодирование: Если данные все же нужно вставить в HTML, кодируйте HTML-сущности (&, <, >). Для атрибутов — свое кодирование. Используйте проверенные библиотеки (например, DOMPurify для HTML).
3. Content Security Policy (CSP): Сильная политика (script-src 'self') может блокировать выполнение вредоносного скрипта даже при успешной XSS-инъекции.

Как разрабатывать безопасные клиентские приложения?

- 4. Безопасные настройки для cookie: HttpOnly, Secure, SameSite.
- 5. Статический анализ кода (SAST): Использование линтеров (ESLint с правилами безопасности) и сканеров кода для поиска опасных паттернов на этапе разработки.

Итоги и ключевые мысли

С ростом сложности клиентских приложений DOM-безопасность выходит на первый план. DOM-based уязвимости часто невидимы для серверных сканеров, требуя специализированных методик и инструментов.

Понимание потока данных — фундаментальный навык для тестировщика клиентской безопасности.

Burp Suite DOM Invader — мощный инструмент, автоматизирующий рутинную часть работы.

Защита — это комбинация: безопасное кодирование на стороне разработчика, политики безопасности (CSP) и регулярное ручное тестирование с упором на клиентскую логику.